Deep Residual Learning for Image Recognition

Kaiming He[1], Xiangyu Zhang[1], Shaoqing Ren[1], Jian Sun[1]
[1]Microsoft Research

According to the original paper, training very deep neural networks has been challenging mainly due to two difficulties: (i) the *vanishing/exploding gradient problem[1, 2]*, where gradient magnitudes diverge as they are back-propagated through many layers, hindering effective training of early layers; and (ii) the *degradation problem*, where the training accuracy of deeper networks first saturates and then degrades,leading to higher training error[3], even though in theory they should perform no worse than their shallower counterparts (since deeper models can represent the shallow solution via identity mappings).

To address the degradation problem, the authors proposed a *deep residual learning framework*. In this framework, instead of forcing the layers to learn the desired mapping $H(x)$ directly, the layers are reformulated to learn a residual function:

$$F(x) = H(x) - x,$$

so that the final output becomes

$$y = F(x) + x.$$

where:

- x: the input to the residual block
- y: the final output of the residual block
- $H(x)$: the target mapping the block aims to approximate
- $F(x)$: the residual function, defined as the difference between the desired mapping and the input

The authors explain that if the identity mapping is optimal, the residual function can be driven toward zero, making it easier to approximate identity mappings using a stack of nonlinear layers.

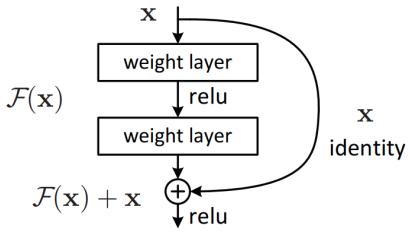

Figure 1: Residual learning: a building block (adapted from He et al., 2015).

They further describe that this formulation relies on *shortcut connections* (Figure 1), which bypass one or more layers by directly adding the input x to the output of the residual function $F(x)$. These shortcuts introduce no additional parameters or computation, which makes them attractive compared to traditional methods. In cases where the dimensions of x and $F(x)$ do not match (e.g., when the number of channels increases), a *projection shortcut* is employed to align dimensions:

$$y = F(x, \{W_i\}) + W_s x,$$

where W_s is typically implemented as a 1×1 convolution. The authors note that although such projections are possible, their experiments showed that identity shortcuts are usually sufficient to solve the degradation problem and are more economical. Thus, W_s is only applied when dimension matching is required.

The authors also state that the residual function $F(x)$ can be composed of two or three layers, although more are possible, while a single-layer form behaves like a linear layer and offers no observed advantage. They emphasize that although their notation uses fully connected layers for simplicity, the formulation extends naturally to convolutional layers, where $F(x, \{W_i\})$ may include multiple convolutions and the skip connection is realized by channel-wise element-wise addition.

They also compared two models for ImageNet such as the plain network (baseline) and the residual network (ResNet). Upon the comparation, the authors report that while the plain network follows a VGG-style[7]design using mainly 3×3 convolutions, equal filter counts for the same feature map size, and filter doubling when the feature map size is halved, it suffers from optimization difficulties as depth increases. In contrast, the residual network augments this baseline with shortcut connections: identity shortcuts are used when input and output dimensions match, and either zero-padding or 1×1 projection shortcuts are used when dimensions differ. Both networks have 34 layers and similar computational cost (about 3.6 billion FLOPs, much smaller than VGG-19's 19.6 billion), but the residual version has fewer filters and lower complexity [7].

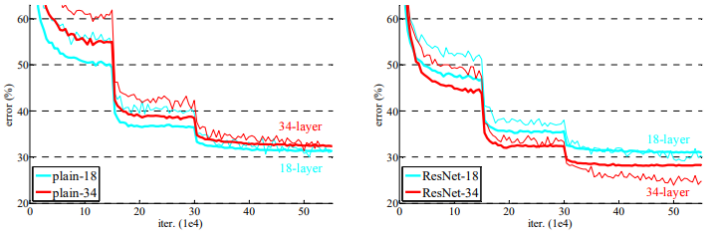

Figure 2: Training on ImageNet. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts

Their models were also evaluated on the ImageNet 2012 classification dataset[6] with 1,000 classes. They found that deeper plain networks (34 layers) performed worse than shallower ones (18 layers), confirming the degradation problem despite the use of batch normalization as it can be seen in Fig. 4 (left). In contrast, residual networks reversed this trend: a 34-layer ResNet achieved significantly lower training error and higher validation accuracy than its plain counterpart.

Furthermore, the authors introduced a bottleneck design for deeper ResNets, where each residual block uses three layers (1×1, 3×3, 1×1 convolutions). The 1×1 layers reduce and then restore dimensions, making the 3×3 layer a computationally efficient bottleneck. They emphasize that identity shortcuts are especially important in this design, since replacing them with projection shortcuts would double the time complexity and model size. It was also reported that the 34-layer ResNets already achieve competitive accuracy, while the 152-layer ResNet reaches a single-model top-5 validation error of 4.49 percent, surpassing all previous ensemble results. An ensemble of six ResNets further reduced the top-5 error to 3.57 percent on the ImageNet test set, setting a new state of the art.

They also conducted more studies on the CIFAR-10 dataset[5] with 50k training images and 10k testing images in 10 classes. Analysis showed that ResNets improved steadily as depth increased, with the 110-layer ResNet achieving 6.43 percent error, being amongn the state-of-the-art results at the time—using fewer parameters than comparable models[4].

The authors adapted Faster R-CNN for detection using ResNet-50/101 models pre-trained on ImageNet and fine-tuned on detection datasets. Unlike VGG-16, ResNets have no fully connected layers, so they applied the "Networks on Convolutional feature maps" (NoC) approach: early convolutional layers (up to conv4) are shared for region proposals and detection, while later conv5 layers act like VGG's fully connected layers. Batch Normalization layers are fixed after pre-training, turning them into linear activations to save memory during fine-tuning.

[1] Y. Bengio, P. Simard, and P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994. doi: 10.1109/72.279181.

[2] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In Yee Whye Teh and Mike Titterington, editors, *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pages 249–256, Chia Laguna Resort, Sardinia, Italy, 13–15 May 2010. PMLR. URL https://proceedings.mlr.press/v9/glorot10a.html.

[3] Kaiming He and Jian Sun. Convolutional neural networks at constrained time cost, 2014. URL https://arxiv.org/abs/1412.1710.

[4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015. URL https://arxiv.org/abs/1512.03385.

[5] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical Report TR-2009, University of Toronto, 2009. URL https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf.

[6] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. Imagenet large scale visual recognition challenge, 2015. URL https://arxiv.org/abs/1409.0575.

[7] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition, 2015. URL https://arxiv.org/abs/1409.1556.